Performance Evaluation of RAG Systems:

How do different RAG pipeline configurations affect end-to-end system performance?

Shirin Manzari

Software Evaluation – Dr. HaroonAbadi

Oct 2026 (V1.0)



IAU, Central Tehran Branch

Related work and this study

Paper	Publisher	Main focus	Connection to this study
RAGPerf (2026)	Proceedings of the VLDB Endowment, 2026	Modular RAG benchmarking	Measure stages and the complete pipeline
RAGO (2025)	ACM, ISCA 2025	RAG workload and serving optimization	Identify stage costs and latency variation
RAG-Stack (2026)	arXiv preprint, 2026	Quality–performance configuration search	Compare configurations and trade-offs
RAGCache (2024)	ACM Transactions on Computer Systems (TOCS); first preprint 2024	Caching retrieved knowledge states	-
CacheBlend (2024/2025)	ACM, EuroSys 2025; first preprint 2024	Reuse and fusion of chunk KV caches	-

Retrieval-Augmented Generation

An approach that combines a pre-trained language model's internal knowledge with externally retrieved information to generate more informed and factual responses.

Lewis et al. (2020)



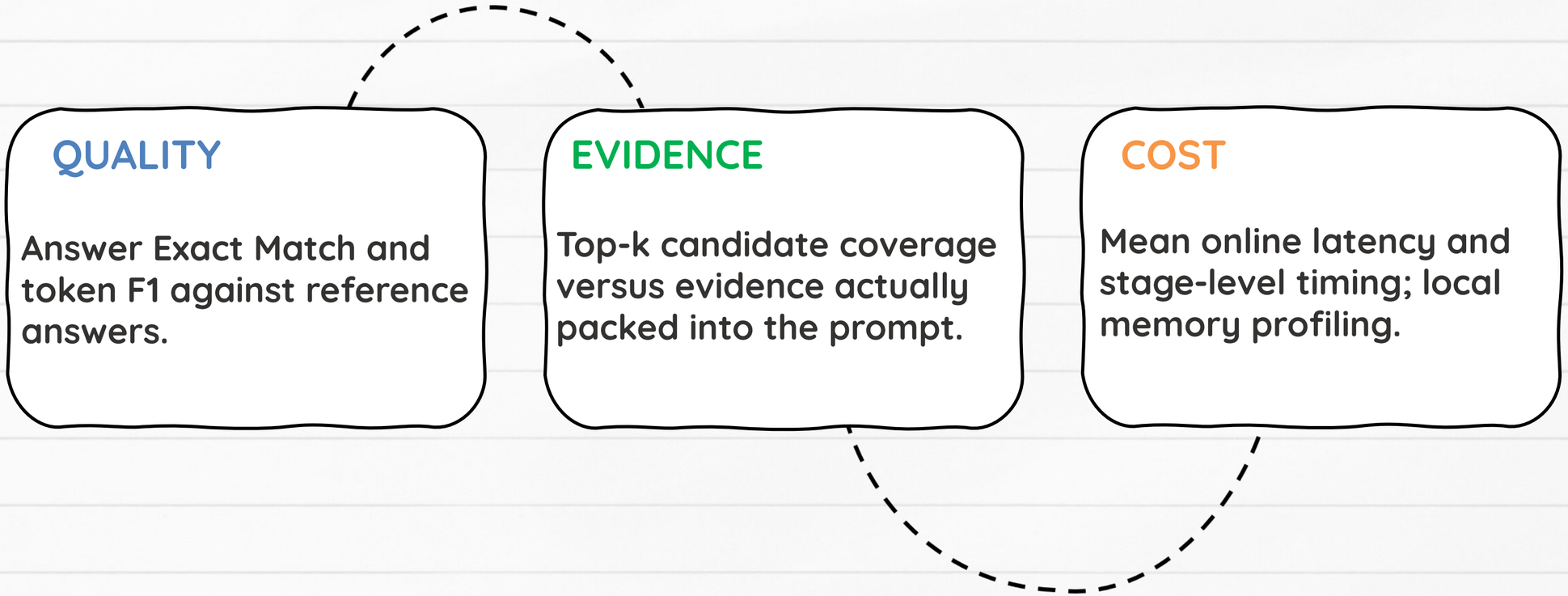
LLM with RAG context



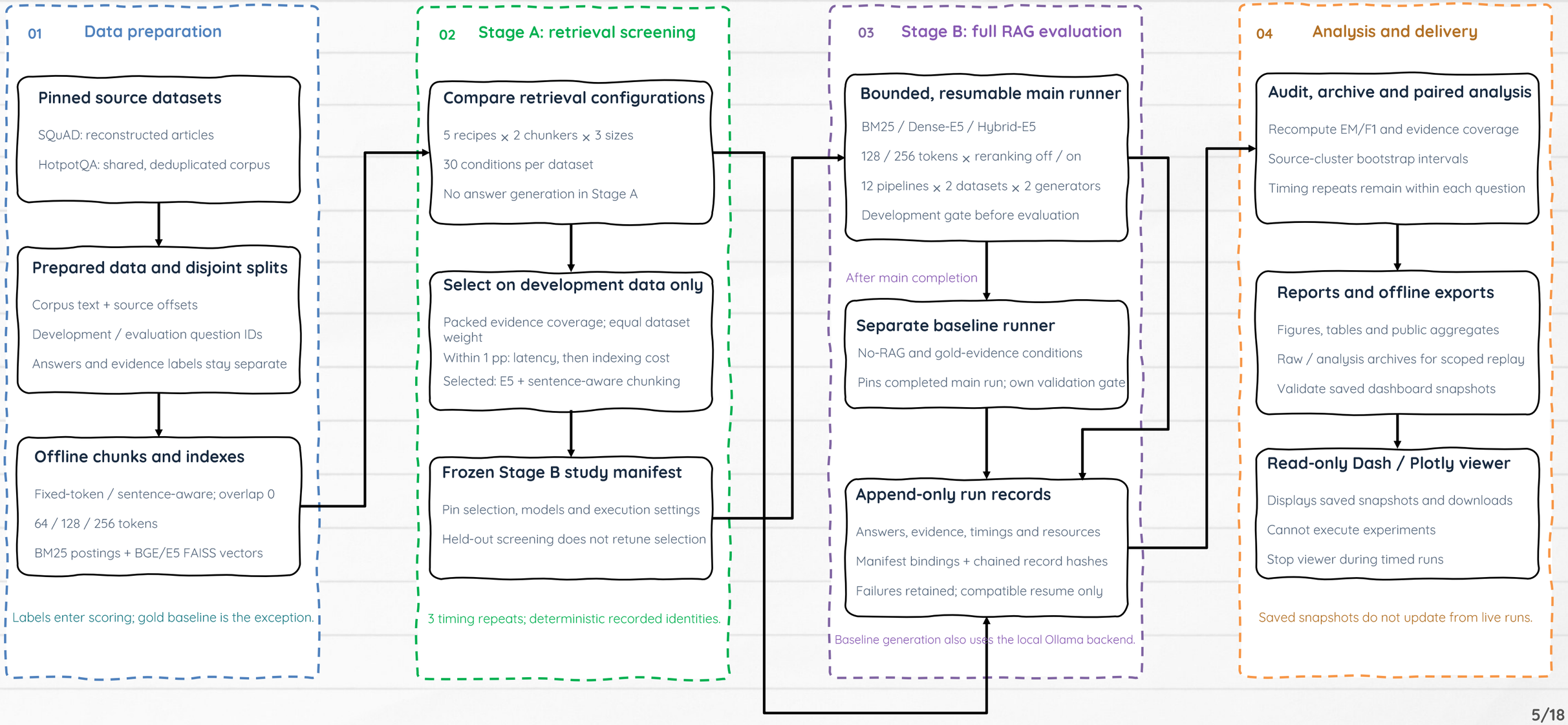
LLM by itself

Research question & evaluation targets

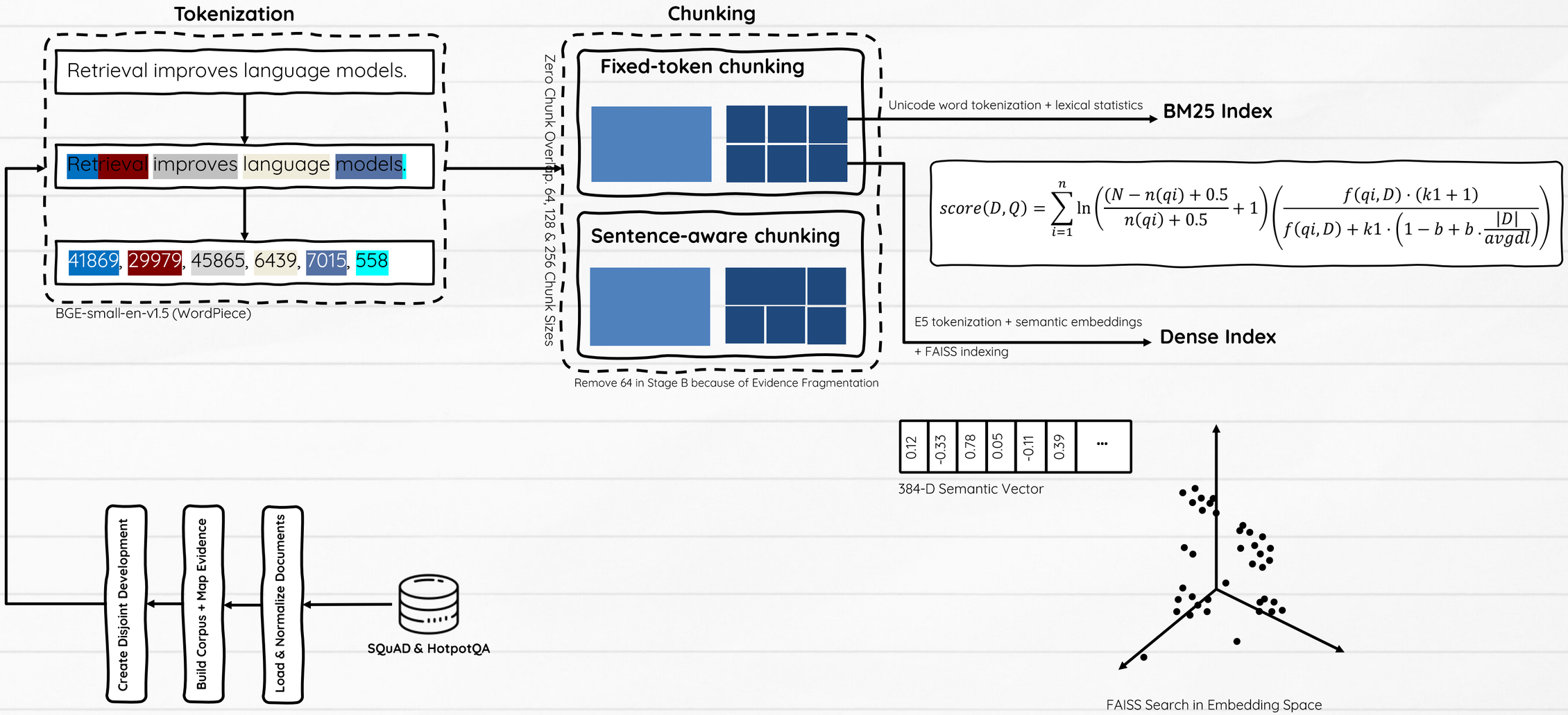
How do retrieval configuration choices change evidence availability, answer quality, and latency?



RAGBench Lab Architecture



Chunking & Indexing



Stage A Configuration

30 unique retrieval configurations evaluated across two datasets, resulting in 60 dataset-specific configurations.

A. Retrieval Methods

Method	Description	Technology
BM25	Lexical matching based on word statistics	BM25
Dense-BGE	Semantic vector similarity	BGE-small-en-v1.5 + FAISS
Dense-E5	Semantic vector similarity	E5-small-v2 + FAISS
Hybrid-BGE	Combines BM25 and Dense-BGE rankings	RRF
Hybrid-E5	Combines BM25 and Dense-E5 rankings	RRF

B. Chunking Strategies: Fixed-token & Sentence-aware

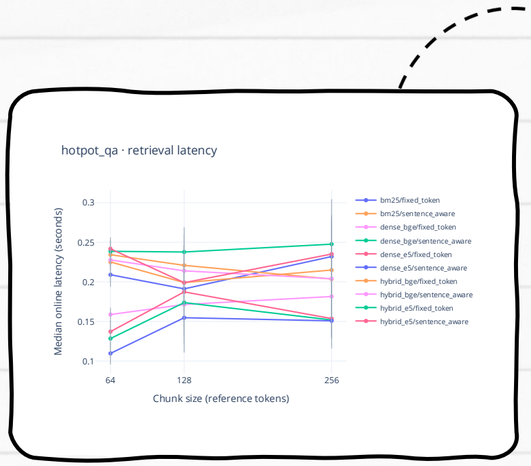
C. Chunk Sizes: 64, 128, 256

D. 50 questions across two datasets: SQuAD & HotpotQA

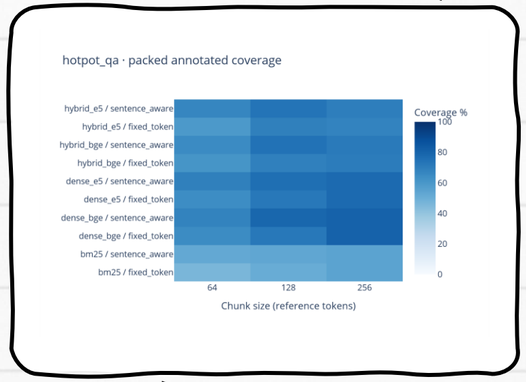
E. Timing repetitions: 3 per configuration and question

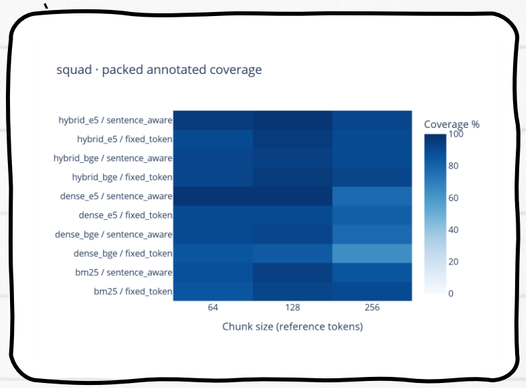
Key Findings & Conclusion

- E5 with sentence-aware chunking achieved the highest evidence coverage (82.82%).
- BGE with sentence-aware chunking ranked second (81.24%).
- Fixed-token chunking performed worse with both E5 (79.08%) and BGE (77.74%).
- Conclusion: E5 with sentence-aware chunking was selected for Stage B, retaining BM25, Dense-E5, and Hybrid-E5 for end-to-end evaluation.









Stage B Configuration

12 unique RAG pipeline configurations evaluated across two datasets and two generators, resulting in 48 experimental conditions.

A. Retrieval Methods

Method	Description	Technology
BM25	Lexical matching based on word statistics	BM25
Dense-E5	Semantic vector similarity	E5-small-v2 + FAISS
Hybrid-E5	Combines BM25 and Dense-E5 rankings	RRF

B. Chunking Strategies: Sentence-aware (fixed)

C. Chunk Sizes: 128, 256

D. Across two datasets: SQuAD & HotpotQA

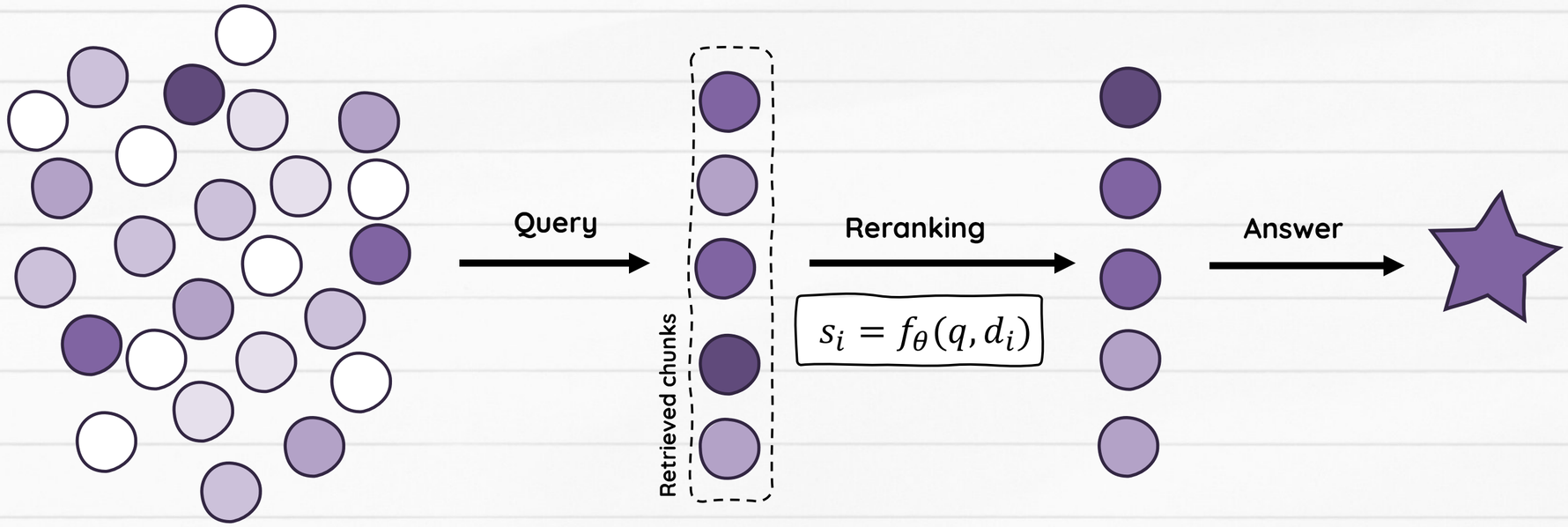
E. Reranking: On & Off

F. Timing repetitions: 3 per configuration and question

G. LLM Answer Generation: Qwen2.5-3B-Instruct & Llama-3.2-3B-Instruct

Reranking

Reranking is the first new experimental component in Stage B. The project uses MS MARCO MiniLM-L6-v2 Cross-Encoder. A Cross-Encoder jointly processes the question and each retrieved chunk to estimate relevance.



Context Packing

After retrieval and optional reranking, the system must decide which evidence chunks to include in the LLM prompt.

The project uses a fixed reference-token budget: $B = 1024\ tokens$

Chunk size	Approximate chunks fitting in
128 tokens	8
256 tokens	4

LLM Generators

Qwen2.5-3B-Instruct

Llama-3.2-3B-Instruct



Generator	Parameters	Serving
Qwen2.5-3B-Instruct	3B	Ollama
Llama-3.2-3B-Instruct	3B	Ollama

Evaluation Metrics

Answer Quality

Exact Match (EM) checks whether the normalized generated answer exactly matches the reference.

$$EM(a, g) = \begin{cases} 1 & \text{norm}(a) = \text{norm}(g) \\ 0 & \text{otherwise} \end{cases}$$

Token F1 measures the overlap between predicted and reference answer tokens.

$$F_1 = \frac{2PR}{P + R}$$

Evidence Coverage

Retrieved Evidence Coverage: Was the required evidence among the top 50 candidates?

Packed Evidence Coverage: Was the required evidence actually included in the LLM prompt?

Latency

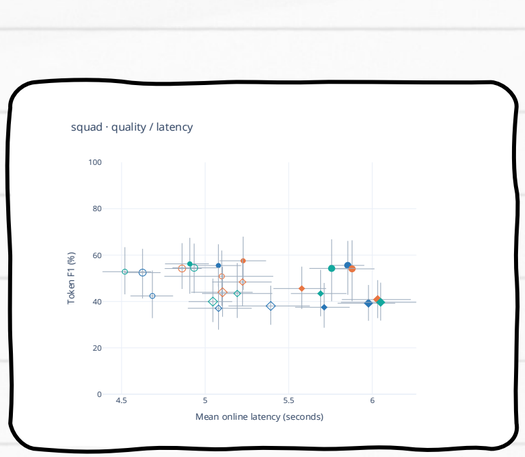
The pipeline measures timing across different components.

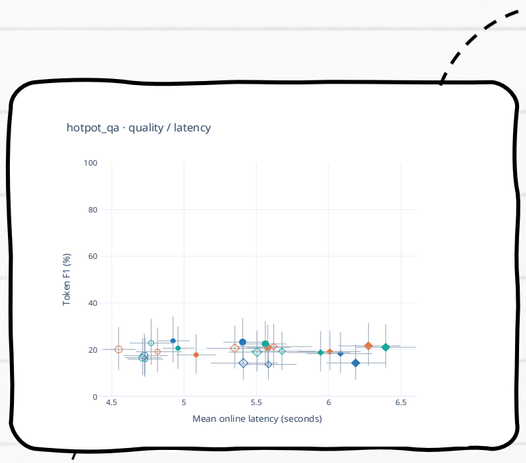
Conceptually:

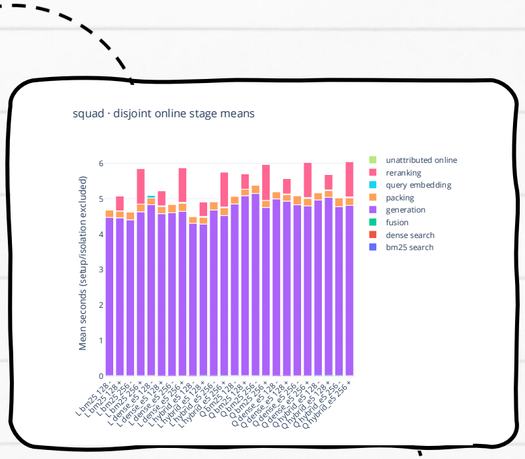
$$T_{\text{total}} \approx T_{\text{retrieval}} + T_{\text{reranking}} + T_{\text{packing}} + T_{\text{generation}} + T_{\text{overhead}}$$

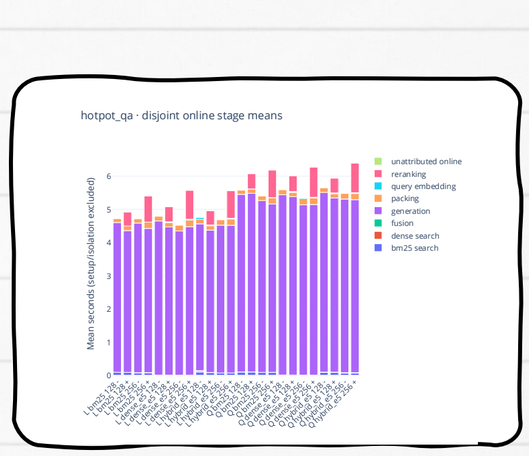
Key Findings & Conclusion

- Llama achieved higher answer quality than Qwen, with a maximum Token F1 of 57.54% on SQuAD.
- SQuAD outperformed HotpotQA, with maximum Token F1 scores of 57.54% and 23.75%, respectively.
- Reranking increased latency by 0.13–1.23 seconds, improving answer quality in only 14 of 24 comparisons.
- Conclusion: No single RAG configuration consistently achieved the best quality and latency, highlighting the importance of end-to-end evaluation.

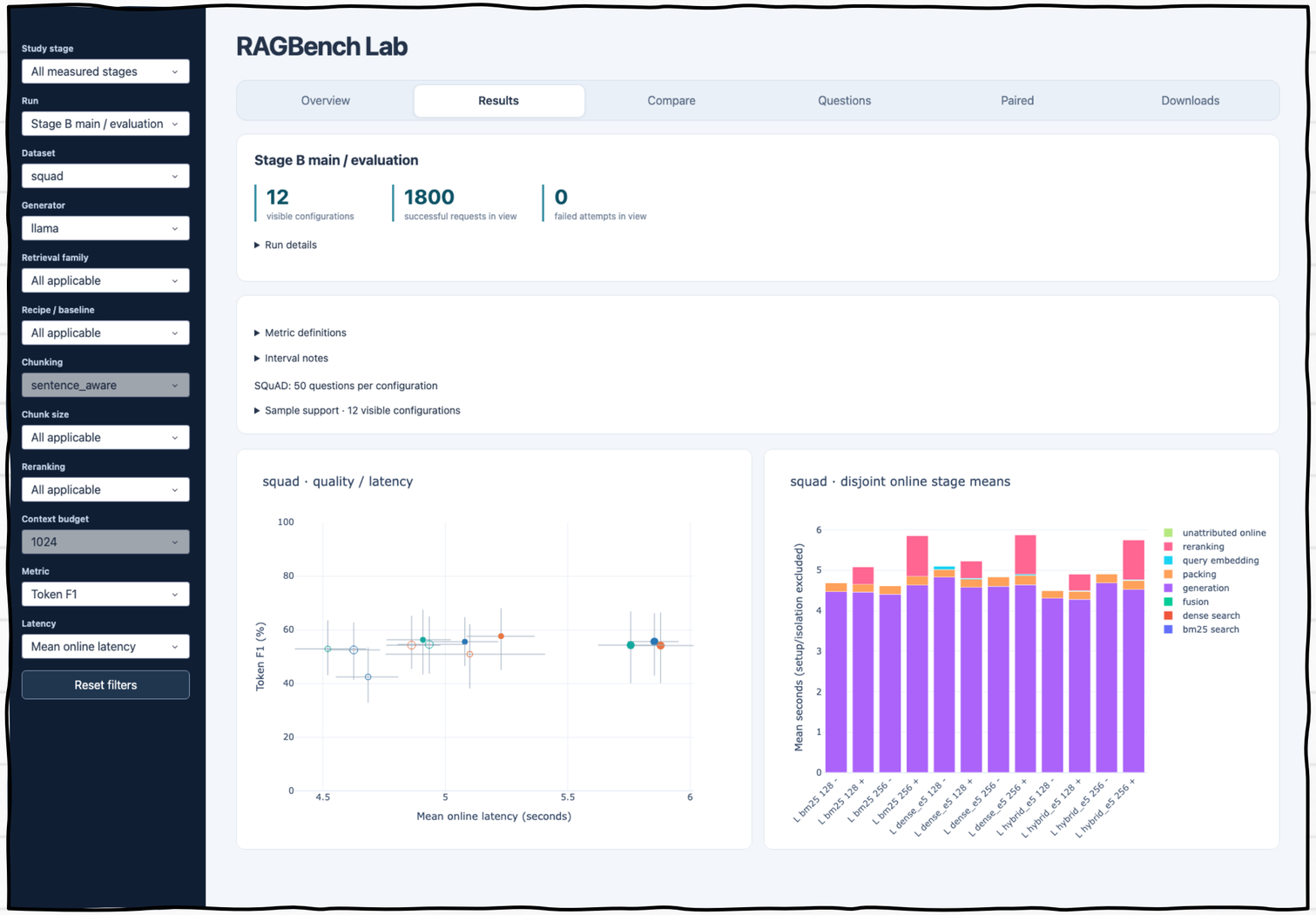








Dashboard Demo & Final Results

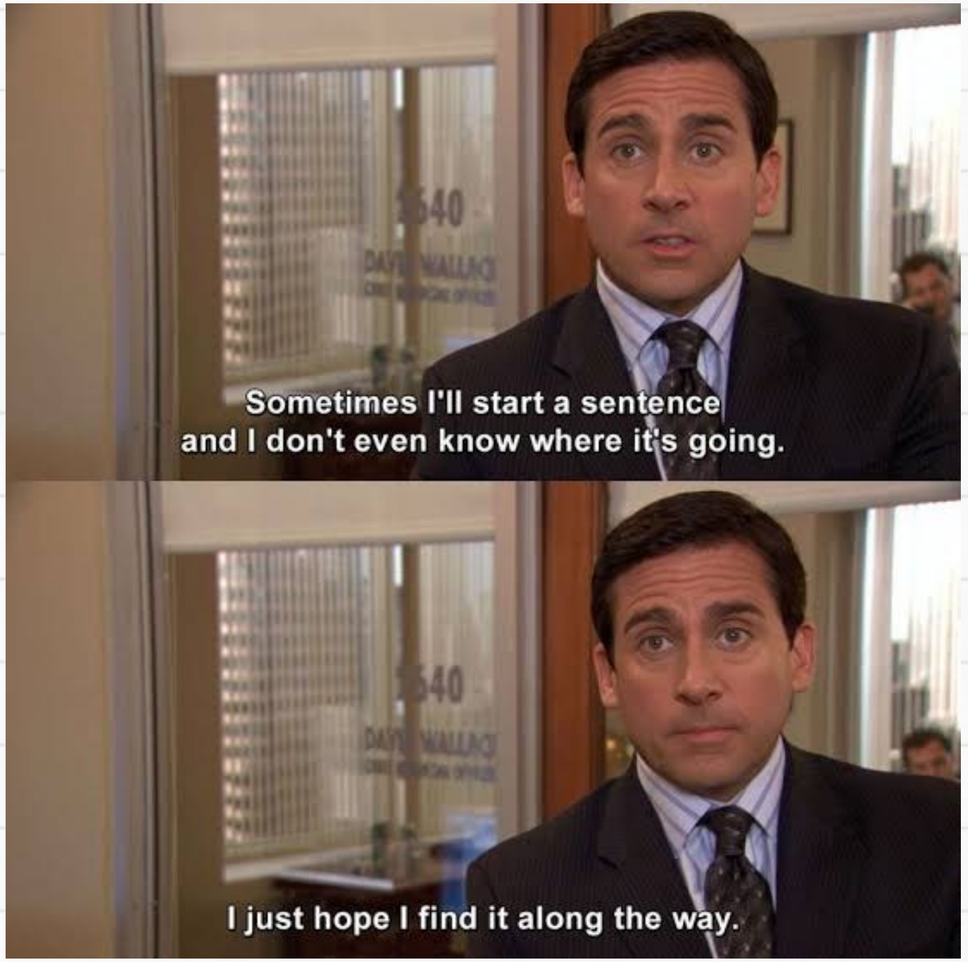


Conclusion

No universal winning RAG pipeline emerged.

1. Reranking consistently increased latency, but its effect on answer quality varied across configurations.
2. Retrieval and context packing were separate sources of evidence loss.
3. Dataset and generator choices significantly influenced the observed quality–latency trade offs.
4. No universally optimal RAG configuration emerged; reproducible, end-to-end evaluation is essential.

So, What's next?



Reference

- Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459–9474.
- Jiang, W., et al. (2025). RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving. Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA), 974–989. ACM. dblp.org
- Li, S., et al. (2026). RAGPerf: An End-to-End Benchmarking Framework for Retrieval-Augmented Generation Systems. Proceedings of the VLDB Endowment, 18(11), 3689–3703. dblp1.uni-trier.de
- Zhang, H., et al. (2026). RAG-Stack: Co-Optimizing RAG Serving Performance and Quality. arXiv preprint arXiv:2608.03487. arxiv.org
- Jin, C., et al. (2026). RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. ACM Transactions on Computer Systems, 44(1), Article 2, 1–27. dblp.dagstuhl.de
- Yao, J., et al. (2025). CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. Proceedings of the Twentieth European Conference on Computer Systems (EuroSys), 94–109. ACM

Tools & Source

Programming Language: Python

Deep Learning & Model Inference: PyTorch and Hugging Face Transformers

Tokenization: Hugging Face Tokenizers, BGE WordPiece Tokenizer for chunking, and Unicode word tokenization for BM25

Chunking Strategies: Fixed-token and sentence-aware chunking (64, 128, and 256 tokens)

Retrieval Methods: BM25, Dense Retrieval, and Hybrid Retrieval using Reciprocal Rank Fusion (RRF)

Vector Search: FAISS CPU (exact inner-product search)

Numerical Computing: NumPy

Local LLM Serving: Ollama

Dashboard & Visualization: Dash and Plotly

Datasets: SQuAD v1.1 and adapted shared-corpus HotpotQA

Embedding Models: BGE-small-en-v1.5 and E5-small-v2

Reranking Model: MS MARCO MiniLM-L6-v2 Cross-Encoder

Generation Models: Qwen2.5-3B-Instruct and Llama-3.2-3B-Instruct



RAGBench Lab on GitHub

https://github.com/shirin-manzari/ragbench-lab